

BOOK 21 — SYSTEM DESIGN / HIGH LEVEL DESIGN (HLD)

Scaling, Data Layer, Caching, Messaging & Reliability at Scale (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 21 of 24 (+1 Special: Book 15A)
Versions Covered: Concept-level (language-agnostic), with Java/Spring Boot grounding drawn from Books 16–17 **Prerequisites:** Book 16 (Microservices), Book 17 (Kafka), Book 19 (LLD) **Next Book:** 22_System_Design_Case_Studies.md

★ **RECRUITER-PRIORITY NOTE:** HLD interviews for senior "Java Full Stack Developer" roles test exactly the vocabulary this series has been building toward — scaling, caching, sharding, async messaging, reliability — but at a **system-wide** level rather than a single service's code. This book deliberately re-uses Book 16/17's Java/Spring-specific mechanics (Eureka, Config Server, Resilience4j, Kafka) as concrete grounding for otherwise abstract HLD concepts, so the two levels reinforce each other instead of feeling like separate subjects.

📖 How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 19 (LLD) ఒక్క class కి focus చేసింది. ఈ పుస్తకం **whole system** మీద focus చేస్తుంది — millions of users ని handle చేయాలంటే, ఒక్క server సరిపోదు; data ఒక్క machine లో పట్టదు; network calls fail అవుతాయి. ప్రతి chapter ఒక HLD concept నేర్పి, దాన్ని ఇప్పటికే build చేసిన Books 16/17's concrete Spring/Kafka mechanics తో connect చేస్తుంది.

English: Book 19 (LLD) focused on a single class. This book focuses on the **whole system** — serving millions of users means one server isn't enough, data doesn't fit on one machine, and network calls fail. Each chapter teaches one HLD concept and connects it to the concrete Spring/Kafka mechanics already built in Books 16/17.

🎯 Learning Objectives

1. Apply a repeatable HLD interview approach: requirements → capacity estimation → high-level components → deep dives → trade-offs.
2. Understand scaling (vertical/horizontal), load balancing, and their limits.
3. Choose correctly between SQL/NoSQL, understand CAP theorem, replication, and sharding.
4. Design caching layers (cache-aside/write-through, Redis, CDN) correctly.
5. Understand asynchronous messaging patterns and reliability-at-scale patterns.
6. Connect every HLD concept to the concrete Spring/Kafka implementation already mastered in Books 16–17.

Table of Contents

Ch.	Title
1	Client-Server Architecture & the HLD Interview Approach
2	Scaling: Vertical vs Horizontal, Load Balancing
3	Capacity Estimation & Back-of-the-Envelope Math
4	Data Layer I: SQL vs NoSQL
5	Data Layer II: CAP Theorem & Consistency Models
6	Data Layer III: Replication & Sharding
7	Caching Strategies (Cache-Aside, Write-Through, Redis)
8	CDN & Content Delivery
9	Asynchronous Communication & Messaging Patterns
10	Reliability at Scale (Redundancy, Failover, Rate Limiting)
11	Microservices Patterns at HLD Scale
12	Putting It Together: A Full System Design Walkthrough
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Client-Server Architecture & the HLD Interview Approach

Telugu Explanation

ప్రతి system design interview ఒకే fundamental shape తో మొదలవుతుంది: **client** ఒక request పంపుతుంది, **server(s)** దాన్ని process చేస్తాయి, **data store** state ని persist చేస్తుంది. Book 19's LLD approach లాగే, HLD కి కూడా ఒక repeatable approach ఉంటుంది — కానీ scope class-level నుండి system-level కి మారుతుంది.

Professional English Explanation

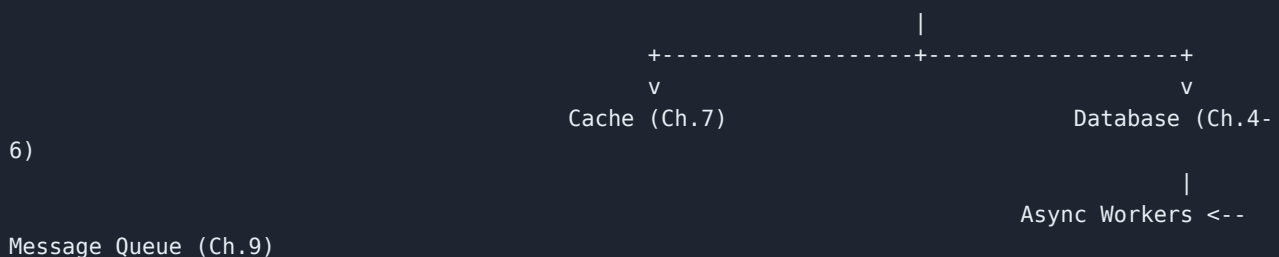
Every system design interview begins with the same fundamental shape: a **client** sends a request, **server(s)** process it, a **data store** persists state. Just like Book 19's LLD approach, HLD has a repeatable approach too — but the scope shifts from class-level to system-level.

The Repeatable HLD Interview Approach (apply this in every chapter that follows)

1. CLARIFY functional requirements (what must the system do?) and NON-functional requirements (how many users? read-heavy or write-heavy? latency needs? consistency needs?)
2. ESTIMATE capacity (Ch.3) - back-of-envelope QPS, storage, bandwidth
3. DEFINE the high-level API (a few key endpoints - reuses Book 12's REST design knowledge)
4. DRAW the high-level component diagram (client -> LB -> services -> cache -> DB -> async workers)
5. DEEP-DIVE into 1-2 components the interviewer cares most about (usually data layer or a hot path)
6. DISCUSS bottlenecks, single points of failure, and how the design scales further
7. DISCUSS trade-offs explicitly - every HLD decision is a trade-off, not a "correct answer"

Diagram — The Universal Starting Point

Client (Book 15A's React app) --> Load Balancer (Ch.2) --> [App Server instances] (Book 16's microservices)



Internal Working

- **Functional vs non-functional requirements** is the first and most commonly skipped distinction: functional requirements describe *what* the system does (post a tweet, place an order); non-functional requirements (scale, latency, consistency, availability) determine *which architecture* is even viable — two systems with identical functional requirements can need completely different architectures if one serves 100 users and the other serves 100 million.

- Deliberately **not** deep-diving into every component is itself a skill — an HLD interview rewards depth on the 1-2 components that matter most for the specific problem (e.g., the data layer for a system with complex consistency needs, or the caching layer for a read-heavy feed), not shallow coverage of everything.
- This chapter's diagram is intentionally the **same shape** as Book 16's microservices architecture and Book 12's layered Spring Boot architecture — HLD generalizes patterns already built concretely in earlier books to arbitrary scale, rather than introducing an unrelated new vocabulary.

Interview Answer

"I approach every HLD problem the same way: clarify both functional and non-functional requirements first, since non-functional requirements (scale, latency, consistency needs) actually determine which architecture is viable. Then I estimate capacity, sketch a high-level API, draw the high-level component diagram, and deliberately deep-dive into only the one or two components most relevant to this specific problem's hard constraints, discussing bottlenecks and trade-offs explicitly rather than presenting one 'correct' design."

Coding Exercise

L1: Write functional and non-functional requirements for a URL shortener. **L2:** Draw the high-level component diagram for a simple blogging platform. **L3:** Identify which 1-2 components deserve a deep dive for a read-heavy news feed vs a write-heavy logging system. **L4 (Interview):** Walk through your HLD interview approach in under 90 seconds. **L5 (Senior):** Given a vague prompt ("design Twitter"), write 5 clarifying questions before drawing anything. **L6 (Mastery):** Apply this 7-step approach, from memory, to a system not covered in this book (e.g., a ride-sharing dispatch system).

CHAPTER 2 — Scaling: Vertical vs Horizontal, Load Balancing

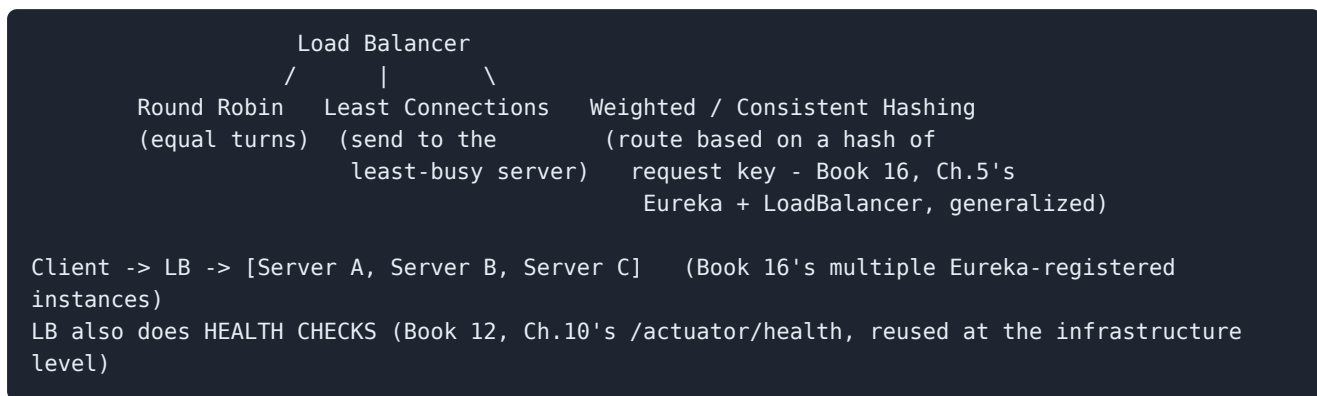
Telugu Explanation

Vertical scaling (scale up) — ఒక్క server కి ఎక్కువ CPU/RAM add చేయడం; సులభం కానీ hardware limit ఉంటుంది, మరియు అది single point of failure గా ఉంటుంది. **Horizontal scaling** (scale out) — ఎక్కువ servers add చేయడం; Book 16's multi-instance microservices ఇక్కడ నుండే వస్తాయి. **Load Balancer** requests ని ఈ multiple instances మధ్య distribute చేస్తుంది.

Professional English Explanation

Vertical scaling (scale up) — adding more CPU/RAM to one server; simple, but has a hardware ceiling and remains a single point of failure. **Horizontal scaling** (scale out) — adding more servers; this is exactly where Book 16's multi-instance microservices come from. A **Load Balancer** distributes requests across these multiple instances.

Diagram — Load Balancing Algorithms



Internal Working

- Horizontal scaling requires the application to be **stateless** — this is precisely why Book 14's `SessionCreationPolicy.STATELESS` and JWT-based auth matter at HLD scale: if session state lived only in one server's memory, a load balancer routing a user's next request to a *different* server would lose that state entirely.
- Consistent hashing** (used by many production load balancers and caches, Ch.7) solves a subtle problem plain modulo-based hashing has: when a server is added or removed, modulo hashing ($\text{hash}(\text{key}) \% N$) remaps almost every key to a different server, causing a massive cache-miss storm; consistent hashing remaps only a small fraction of keys, minimizing disruption.
- Health checks** are what let a load balancer stop routing to a server that's up but unhealthy (e.g., its database connection pool is exhausted) — this directly reuses Book 12, Ch.10's Actuator health endpoint concept, now consumed by infrastructure rather than a human operator.

Real-World Example

CDN providers and large-scale caching layers (Ch.7-8) rely heavily on consistent hashing specifically to avoid the cache-miss storm that would occur if adding one new cache server invalidated most of the existing cache's key-to-server mapping.

Interview Answer

"Vertical scaling adds resources to one machine but hits a hardware ceiling and remains a single point of failure; horizontal scaling adds more machines and requires the application to be stateless, which is why JWT-based stateless authentication (Book 14) matters at scale — session state pinned to one server breaks under load-balanced routing. Load balancers use algorithms like round robin, least connections, or consistent hashing; consistent hashing specifically minimizes remapping when servers are added or removed, avoiding the cache-miss storm that simple modulo hashing would cause."

Cross Questions

- Q: Why must an application be stateless to scale horizontally? → A: A load balancer can route a user's requests to any instance; if session state lived only in one instance's memory, requests routed elsewhere would lose access to that state.
- Q: What problem does consistent hashing solve that plain modulo hashing ($\text{hash}(\text{key}) \% N$) doesn't? → A: Modulo hashing remaps nearly all keys when N (server count) changes; consistent hashing remaps only a small fraction, avoiding a massive cache-miss storm on scaling events.

Tricky Questions

- Q: Is horizontal scaling always preferable to vertical scaling? → A: Not always — vertical scaling is simpler operationally and avoids distributed-system complexity entirely; it's often the right first step until its ceiling is actually reached, rather than defaulting to horizontal scaling prematurely.

Coding Exercise

L1: Compare vertical vs horizontal scaling trade-offs for a database vs a stateless API server. **L2:** Explain why round-robin load balancing can be suboptimal for requests with very different processing costs. **L3:** Trace what happens to key-to-server mapping when a 4th server is added, under modulo hashing vs consistent hashing. **L4 (Interview):** Explain why statelessness is a prerequisite for horizontal scaling. **L5 (Senior):** Design a load-balancing strategy for a system with both cheap read requests and expensive write requests. **L6 (Mastery):** Explain consistent hashing's ring structure and virtual nodes concept in full.

CHAPTER 3 — Capacity Estimation & Back-of-the-Envelope Math

Telugu Explanation

HLD interview లో "design Twitter" అంటే, మొదట numbers అంచనా వేయాలి — daily active users, QPS (queries per second), storage per day/year, bandwidth. ఇది ఖచ్చితమైన numbers కోసం కాదు — ఇది **which architecture decisions actually matter** అని నిర్ణయించడానికి.

Professional English Explanation

In an HLD interview, "design Twitter" first requires estimating numbers — daily active users, QPS (queries per second), storage per day/year, bandwidth. This isn't about precise numbers — it's about determining **which architecture decisions actually matter**.

The Estimation Framework

```
GIVEN: 100 million daily active users, each posts 2 times/day, reads 50 times/day

WRITE QPS = (100M users x 2 posts) / 86,400 seconds/day ≈ 2,300 writes/sec
READ QPS  = (100M users x 50 reads) / 86,400 seconds/day ≈ 58,000 reads/sec
  -> READ-HEAVY by a factor of ~25x -> THIS is the number that justifies Ch.7's caching layer

STORAGE (posts only, avg 200 bytes/post):
  100M users x 2 posts/day x 200 bytes = 40 GB/day -> ~14.6 TB/year
  -> at this scale, a single database server's disk is a real constraint -> justifies Ch.6's sharding

BANDWIDTH (reads, avg 200 bytes/post):
  58,000 reads/sec x 200 bytes ≈ 11.6 MB/sec -> modest, not the primary bottleneck here
```

Internal Working

- The **read:write ratio** derived here (25:1) is the single most architecturally consequential number in this estimation — it directly justifies investing heavily in caching (Ch.7) and read replicas (Ch.6) rather than optimizing the write path, and an interviewer expects this number to visibly *drive* subsequent design decisions, not just sit unused after being calculated.
- Estimates are deliberately **rounded to one significant figure** (100 million, not 97.3 million) — the goal is order-of-magnitude reasoning fast enough to guide design decisions, not spreadsheet-level precision; spending 10 minutes on precise arithmetic in an interview is a common, costly mistake.
- Storage estimation revealing "~14.6 TB/year" is what justifies moving from "a single PostgreSQL instance" to "sharding is required" (Ch.6) — without this number explicitly computed, sharding looks like premature over-engineering rather than a data-driven necessity.

Real-World Example

Engineering teams at scale routinely perform exactly this kind of estimation before choosing infrastructure — a read:write ratio and projected storage growth directly inform decisions like "do we need a caching layer," "do we need to shard now or in 18 months," long before any code is written.

Interview Answer

"I estimate capacity using round, order-of-magnitude numbers — daily active users, actions per user per day, converted to QPS by dividing by 86,400 seconds. The read:write ratio this reveals is usually the single most important number, since it directly justifies whether caching and read replicas are worth the added complexity. Storage estimates over a year similarly reveal whether a single database instance is even viable, which is what justifies sharding as a data-driven decision rather than premature optimization."

Cross Questions

- Q: Why round estimates to one significant figure instead of calculating precisely? → A: The goal is fast, order-of-magnitude reasoning that drives architecture decisions, not spreadsheet precision — precise arithmetic wastes interview time without changing which architecture is justified.
- Q: What did the read:write ratio calculation in this chapter directly justify? → A: Investing in caching (Ch.7) and read replicas (Ch.6), since reads outnumber writes by roughly 25 to 1.

Tricky Questions

- Q: Is capacity estimation just a formality before "the real design work"? → A: No — a strong HLD answer visibly uses these numbers to justify specific decisions (caching, sharding, which components need redundancy); estimation disconnected from the rest of the design is a common, weak pattern interviewers penalize.

Coding Exercise

L1: Estimate QPS for a system with 10 million daily users, each performing 5 read and 1 write actions daily. **L2:** Estimate yearly storage for a photo-sharing app with 1 million daily uploads averaging 2MB each. **L3:** Given your Ch.2 estimates, decide whether horizontal scaling is justified yet. **L4 (Interview):** Walk through a full capacity estimation for a chat application in under 5 minutes. **L5 (Senior):** Use a computed read:write ratio to justify three specific downstream architecture decisions. **L6 (Mastery):** Estimate capacity for a video-streaming platform, addressing both storage and the very different bandwidth profile of video versus text/photo data.

CHAPTER 4 — Data Layer I: SQL vs NoSQL

Telugu Explanation

Book 09/13 లో SQL (relational, JDBC/JPA) వాడారు. HLD లో scale పెరిగే కొద్దీ, **NoSQL** databases (document/key-value/wide-column/graph) కూడా options అవుతాయి. ఎంపిక ఆధారపడింది: data relationships ఎంత complex గా ఉన్నాయి, consistency ఎంత strict గా అవసరం, scale ఎంత అవసరం అనే దానిమీద.

Professional English Explanation

Books 09/13 used SQL (relational, JDBC/JPA). At HLD scale, **NoSQL** databases (document/key-value/wide-column/graph) also become options. The choice depends on: how complex the data relationships are, how strict consistency needs to be, and how much scale is required.

Diagram — Database Type Decision Guide

```
Complex relationships, joins, ACID transactions needed?
-> YES -> SQL (PostgreSQL/MySQL) - Book 09/13's relational model, JOINS, foreign keys

Simple key -> value lookups, extreme scale, low latency?
-> YES -> Key-Value Store (Redis - Ch.7, DynamoDB)

Semi-structured, flexible schema, document-shaped data?
-> YES -> Document Store (MongoDB) - no rigid schema, easy horizontal scaling

Massive write volume, time-series or wide-column data?
-> YES -> Wide-Column Store (Cassandra) - built for horizontal write scale

Highly connected data, "friends of friends" queries?
-> YES -> Graph Database (Neo4j) - relationship traversal is a first-class operation
```

Internal Working

- SQL databases enforce a **fixed schema** and strong relational integrity (foreign keys, Book 13's entity relationships) — this is exactly what makes complex, multi-entity transactions (Book 13, Ch.7's `@Transactional`) safe and correct, but it's also what makes horizontal scaling (sharding, Ch.6) structurally harder, since joins across shards are expensive or impossible.
- NoSQL databases generally trade some of that relational rigor (joins, cross-entity ACID transactions) for horizontal scalability and schema flexibility — this isn't "NoSQL is worse," it's a genuine trade-off: a document store's denormalized, embedded documents avoid needing joins at all, at the cost of potential data duplication (the same trade-off Book 16, Ch.9's CQRS read models deliberately make).
- Many real production systems use **both** — SQL for the core transactional data (orders, payments, needing ACID) and a NoSQL store for a specific access pattern (session cache, product catalog search, activity feed) — recognizing "this is a polyglot persistence decision, not an either/or" is a senior-level signal.

Real-World Example

E-commerce platforms typically use PostgreSQL/MySQL (Book 09/13) for orders and payments (needing strong consistency and transactions), while using a document store or search-optimized index for product catalog browsing (needing flexible schema and fast full-text search) — exactly the polyglot-persistence pattern.

Interview Answer

"The choice between SQL and NoSQL depends on data relationship complexity, consistency needs, and scale requirements — not a blanket 'NoSQL is more scalable' rule. SQL's fixed schema and relational integrity make complex, ACID-guaranteed transactions across entities safe, which is exactly what Book 13's `@Transactional` relies on, but this rigor makes horizontal sharding harder since cross-shard joins are expensive. NoSQL variants (document, key-value, wide-column, graph) each optimize for a specific access pattern by relaxing some of that relational rigor. Production systems very often use both — polyglot persistence — choosing the right store per access pattern rather than one database for the entire system."

Cross Questions

- Q: Why does SQL's relational integrity make horizontal sharding structurally harder than for many NoSQL stores? → A: Joins and cross-entity ACID transactions become expensive or impossible once related data lives on different shards/machines, whereas many NoSQL models are designed around denormalized, self-contained documents/rows that don't need cross-node joins.
- Q: What real trade-off does a document store's denormalization mirror from earlier in this series? → A: Book 16, Ch.9's CQRS read models — both accept data duplication in exchange for avoiding expensive joins/queries at read time.

Tricky Questions

- Q: Does choosing NoSQL mean giving up consistency entirely? → A: No — many NoSQL databases offer tunable consistency (e.g., Cassandra's per-query consistency level) rather than an all-or-nothing choice; "NoSQL = eventually consistent always" is an oversimplification worth correcting in an interview.

Coding Exercise

L1: Classify 5 given data access patterns (orders, session cache, social graph, product catalog, time-series metrics) into the appropriate database type. **L2:** Design a polyglot-persistence architecture for an e-commerce platform, naming which store handles which data. **L3:** Identify a scenario where a document store's denormalization would cause a real data-integrity problem. **L4 (Interview):** Explain the SQL vs NoSQL trade-off without saying "it depends" as your entire answer. **L5 (Senior):** Justify using Cassandra specifically for a time-series metrics ingestion system. **L6 (Mastery):** Design the data layer for a social network combining a relational store, a graph store, and a cache, justifying each choice.

CHAPTER 5 — Data Layer II: CAP Theorem & Consistency Models

Telugu Explanation

CAP Theorem: ఒక distributed system ఏకకాలంలో మూడు గ్యారంటీలు — **Consistency** (అన్ని nodes ఒకే data చూపిస్తాయి), **Availability** (ప్రతి request ఒక response పొందుతుంది), **Partition Tolerance** (network split అయినా system పనిచేస్తుంది) — పూర్తిగా ఇవ్వలేదు. Network partitions నిజంగా జరుగుతాయి కాబట్టి, real choice C vs A మధ్యే.

Professional English Explanation

CAP Theorem: a distributed system cannot fully guarantee all three of — **Consistency** (all nodes see the same data), **Availability** (every request gets a response), **Partition Tolerance** (the system keeps working despite a network split) — simultaneously. Since network partitions genuinely happen, the real choice is between C and A.

Diagram — CAP in Practice

Network partition occurs (nodes can't talk to each other):

CP choice (Consistency over Availability):

- > System REFUSES some requests to guarantee no stale/conflicting data is ever returned
- > Example: a banking system's balance check MUST be consistent, even if it means an error

AP choice (Availability over Consistency):

- > System KEEPS RESPONDING, possibly with STALE data, accepting eventual consistency
- > Example: Book 16, Ch.5's Eureka - explicitly AP, prefers a stale registry over no registry at all

Internal Working

- **Partition tolerance isn't optional** in a real distributed system — networks fail; the actual engineering decision is what happens *during* a partition: reject requests to stay consistent (CP) or keep serving, accepting temporary inconsistency (AP) — CAP is really a "pick 2 of 3" only in the sense that P is non-negotiable, making it fundamentally a C-vs-A trade-off.
- This directly explains a decision already made concretely in Book 16, Ch.5: Eureka is explicitly **AP** — it would rather return a possibly-stale list of service instances than refuse to answer during a network partition, because an unavailable service registry would block *all* inter-service communication system-wide, a worse outcome than occasionally routing to a now-dead instance (which Ch.7's Circuit Breaker/Retry then absorbs).
- **Eventual consistency** (the AP choice) doesn't mean "never consistent" — it means all nodes *will* converge to the same state once the partition heals and updates propagate; this is the same underlying idea as Book 16, Ch.8's SAGA pattern and Book 17's Kafka consumer offset model, both of which accept a temporary inconsistency window in exchange for availability/decoupling.

Real-World Example

Amazon's DynamoDB was explicitly designed AP-first (favoring availability, with tunable consistency), based on the observation that for a shopping cart, being able to add an item (even if briefly inconsistent across replicas) matters more than rejecting the request outright during a network hiccup.

Interview Answer

"CAP theorem says a distributed system can't fully guarantee Consistency, Availability, and Partition Tolerance simultaneously — and since partitions are a fact of networked systems, not something you can opt out of, the real engineering decision is CP (reject requests to guarantee consistency) versus AP (keep serving, accepting temporary staleness). Book 16's Eureka is a concrete AP example: it prefers a stale service registry over no registry at all, because unavailability there would block all inter-service communication. Eventual consistency means nodes converge once the partition heals, not that they're never consistent — the same underlying trade-off SAGA (Book 16, Ch.8) and Kafka's consumer model (Book 17) make elsewhere in this series."

Cross Questions

- Q: Why is Partition Tolerance not really a choice in CAP theorem's "pick 2 of 3"? → A: Network partitions are a physical reality in any distributed system; a design that assumes they never happen isn't a valid real-world CP or AP system, so the meaningful trade-off is specifically between C and A.
- Q: Why did Eureka's designers choose AP over CP? → A: Because an unavailable service registry would block ALL inter-service communication system-wide, which is worse than occasionally routing a request to a now-dead instance — a failure mode already handled by Circuit Breaker/Retry (Book 16, Ch.7).

Tricky Questions

- Q: Does choosing AP mean a system can never offer strong consistency for any operation? → A: No — many real systems apply different consistency choices per operation type (e.g., strongly consistent for a payment balance check, eventually consistent for a "likes" counter) rather than one global CAP choice for the entire system.

Coding Exercise

L1: Classify 5 real system behaviors (bank transfer, social media like count, DNS lookup, shopping cart, service registry) as requiring CP or AP. **L2:** Explain why Eureka (Book 16, Ch.5) is AP using CAP vocabulary explicitly. **L3:** Design a system that applies CP for one operation and AP for another within the same application. **L4 (Interview):** Explain CAP theorem and why P isn't really optional. **L5 (Senior):** Justify DynamoDB's AP-first design for a shopping cart use case. **L6 (Mastery):** Explain how Book 16, Ch.8's SAGA pattern and eventual consistency relate to the AP side of CAP theorem.

CHAPTER 6 — Data Layer III: Replication & Sharding

Telugu Explanation

Replication: ఒకే data ని multiple servers me^{లీ}ద copy చేయడం — availability మరియు read scalability కోసం (Book 16, Ch.6's replication factor ఇక్కడ నుండే వస్తుంది). **Sharding:** data ని horizontally partition చేయడం, వేర్వేరు servers me^{లీ}ద వేర్వేరు subset ఉండేలా — write scalability మరియు storage limits అధిగమించడానికి.

Professional English Explanation

Replication: copying the same data across multiple servers — for availability and read scalability (this is exactly where Book 16, Ch.6's replication factor comes from). **Sharding:** horizontally partitioning data so different servers hold different subsets — for write scalability and overcoming storage limits.

Diagram — Replication vs Sharding

REPLICATION (same data, multiple copies):

Primary DB ---replicate---> Replica 1 (read-only)

---replicate---> Replica 2 (read-only)

Writes go to Primary; Reads can be load-balanced across replicas -> solves READ scale (Ch.3's 25:1 ratio)

SHARDING (different data, different servers):

Shard 1: users A-H Shard 2: users I-P Shard 3: users Q-Z

Each shard is a full, independent database holding only ITS slice -> solves WRITE scale + storage limits

Internal Working

- Replication directly addresses Chapter 3's read-heavy workload finding — routing the 58,000 reads/sec across multiple read replicas while writes go only to the primary is the standard first scaling move for a read-heavy system, and it's conceptually identical to Book 16, Ch.6's broker replication (leader handles writes, followers replicate and can serve reads).
- Sharding's hardest design decision is the **shard key** — a poorly chosen key (e.g., sharding by signup date) can create "hot shards" (all recent, active users landing on one shard) while a well-chosen key (e.g., a hash of user ID) distributes load evenly; this exact same "hot partition" concern is why Book 17, Ch.2 warned that Kafka partition-key choice affects load distribution, not just ordering.
- Sharding fundamentally breaks cross-shard joins and transactions (Ch.4's SQL trade-off made concrete) — a query needing data from users on two different shards either requires an application-level join (slow, complex) or a redesign to avoid needing it, which is why sharding is deferred until Ch.3's capacity estimation genuinely justifies it, not applied preemptively.

Real-World Example

Large-scale social platforms shard user data by a hash of user ID specifically to avoid hot shards, while replicating each shard for both read scalability and disaster recovery — combining both techniques, not choosing one over the other.

Interview Answer

"Replication copies the same data across multiple servers, primarily to scale reads and improve availability — directly addressing a read-heavy workload like Chapter 3's example, and conceptually the same mechanism as Book 16's broker replication. Sharding instead partitions different data across servers to scale writes and overcome single-machine storage limits. The hardest sharding decision is choosing a shard key that distributes load evenly and avoids hot shards — the same concern Book 17 raised about Kafka partition keys. Sharding also breaks cross-shard joins, which is why it's a scale-justified decision, not a default."

Cross Questions

- Q: Why does replication primarily solve read scalability rather than write scalability? → A: Writes still need to go to the primary (or be coordinated across replicas) to maintain consistency, but reads can be served from any up-to-date replica, distributing read load across many machines.
- Q: What real problem does a poorly chosen shard key cause? → A: "Hot shards" — uneven load distribution where one shard receives disproportionately more traffic/data than others, the same hot-partition concern Book 17 raised for Kafka.

Tricky Questions

- Q: Does adding more read replicas help a write-heavy workload? → A: No — replicas primarily offload read traffic; a write-heavy bottleneck requires sharding (distributing writes across multiple independent databases) or a different write-optimized data store (Ch.4's wide-column stores), not more read replicas.

Coding Exercise

L1: Design a replication topology (1 primary, 2 replicas) and explain the read/write routing. **L2:** Choose a shard key for a multi-tenant SaaS application's customer data and justify avoiding hot shards. **L3:** Identify a query that would require an expensive cross-shard join, and redesign the data model to avoid it. **L4 (Interview):** Explain the difference between replication and sharding without conflating them. **L5 (Senior):** Design a combined replication + sharding strategy for a 100-million-user system. **L6 (Mastery):** Explain re-sharding (adding a new shard to an already-sharded system) and the operational challenges it introduces.

CHAPTER 7 — Caching Strategies (Cache-Aside, Write-Through, Redis)

Telugu Explanation

Chapter 3's 25:1 read:write ratio నేరుగా justify చేసేది ఇదే — **caching. Cache-Aside** (most common): application cache ని check చేస్తుంది, miss అయితే DB నుండి fetch చేసి cache లో పెడుతుంది. **Write-Through**: ప్రతి write cache మరియు DB రెండింటికీ ఏకకాలంలో వెళ్తుంది — cache ఎప్పుడూ fresh గా ఉంటుంది, కానీ write latency పెరుగుతుంది.

Professional English Explanation

This is exactly what Chapter 3's 25:1 read:write ratio justifies — **caching. Cache-Aside** (most common): the application checks the cache first, and on a miss, fetches from the DB and populates the cache. **Write-Through**: every write goes to both cache and DB simultaneously — the cache stays fresh, at the cost of increased write latency.

Java Code — Cache-Aside Pattern

```
class ProductService {
    private final RedisTemplate<String, Product> redisTemplate; // Spring Data Redis
    private final ProductRepository repository; // Book 13's JPA repository

    Product getProduct(Long productId) {
        String cacheKey = "product:" + productId;
        Product cached = redisTemplate.opsForValue().get(cacheKey); // 1. check cache
FIRST
        if (cached != null) return cached; // CACHE HIT - fast path, no DB call

        Product fromDb = repository.findById(productId) // 2. CACHE MISS
- fall back to DB
        .orElseThrow(() -> new ProductNotFoundException(productId));
        redisTemplate.opsForValue().set(cacheKey, fromDb, Duration.ofMinutes(10)); // 3.
populate cache for NEXT time
        return fromDb;
    }

    void updateProduct(Product product) {
        repository.save(product); // update source
of truth
        redisTemplate.delete("product:" + product.getId()); // INVALIDATE,
don't update, the cache
    }
}
```

Internal Working

- `updateProduct` **deletes** the cache entry rather than updating it directly — this is a deliberate, common best practice: deleting is simpler and avoids a race condition where two concurrent

writes could leave a stale value cached if both tried to update the cache directly; the next `getProduct` call simply repopulates it correctly from the DB.

- The `Duration.ofMinutes(10)` **TTL (time-to-live)** is a safety net against cache entries never being explicitly invalidated (e.g., a direct DB update bypassing the service layer) — this is a deliberate trade-off between cache freshness and cache hit rate, and choosing the TTL requires the same kind of reasoning as Book 16, Ch.7's resilience timeout tuning.
- **Cache invalidation is famously one of the two hardest problems in computer science** — the specific hard case is keeping a cache consistent when the *same* data can be written from multiple paths (a direct SQL update, a batch job, a different service) that don't all know to invalidate the same cache key; production systems address this with event-driven invalidation (an "entity changed" Kafka event, Book 17, triggering cache eviction) rather than hoping every write path remembers to invalidate.

Real-World Example

Nearly every high-traffic e-commerce product page uses exactly this cache-aside pattern with Redis — product details change rarely relative to how often they're viewed, making the 25:1-style read:write ratio (Ch.3) an ideal caching candidate.

Interview Answer

"Cache-Aside is the most common caching pattern: check the cache first, and on a miss, fetch from the database and populate the cache for next time. Writes invalidate (delete) the cache entry rather than updating it directly, avoiding a race condition where concurrent writes could leave stale cached data. A TTL acts as a safety net against invalidation paths that don't explicitly clear the cache. The genuinely hard part of caching at scale is invalidation when the same data can be written through multiple paths that don't all know about the same cache key — production systems often solve this with event-driven invalidation using something like a Kafka event signaling that an entity changed."

Cross Questions

- Q: Why does `updateProduct` delete the cache entry instead of updating it with the new value directly? → A: Deleting avoids a race condition between concurrent writes that could otherwise leave a stale value cached; the next read simply repopulates the cache correctly from the database.
- Q: What safety net does a cache entry's TTL provide? → A: Protection against invalidation paths that bypass the service layer (like a direct DB update or batch job) and would otherwise leave a stale cache entry indefinitely.

Tricky Questions

- Q: Does adding a cache always improve a system's overall reliability? → A: Not automatically — a cache adds a new failure mode (cache unavailable, or serving stale data past an acceptable staleness window) and a new component to operate; it's justified specifically by a favorable read:write ratio and tolerance for eventual consistency, not added by default.

Coding Exercise

L1: Implement `ProductService`'s cache-aside pattern with Redis. **L2:** Implement Write-Through caching for the same service and compare write latency implications. **L3:** Design event-driven cache invalidation using a Kafka event (Book 17) for a multi-service write path. **L4 (Interview):** Explain why cache invalidation (not cache reading) is the hard part of caching. **L5 (Senior):** Choose an appropriate TTL for three different data types (user profile, stock price, static config) and justify each. **L6 (Mastery):** Design a multi-level caching strategy (local in-process cache + Redis + CDN, Ch.8) for a read-heavy product catalog.

CHAPTER 8 — CDN & Content Delivery

Telugu Explanation

Ch.7's cache application server కి దగ్గరగా ఉంటుంది. **CDN (Content Delivery Network)** ఇంకా ఒక అడుగు ముందుకు వెళ్ళి, static content (images, videos, JS/CSS — Book 15A's React build output) ని **user కి geographically దగ్గరగా** ఉన్న edge servers లో cache చేస్తుంది — origin server కి request వెళ్ళాల్సిన అవసరమే లేకుండా.

Professional English Explanation

Chapter 7's cache lives close to the application server. A **CDN (Content Delivery Network)** goes one step further, caching static content (images, videos, JS/CSS — Book 15A's React build output) on **edge servers geographically close to the user** — avoiding the need for the request to ever reach the origin server.

Diagram — CDN Request Flow

```
User in Mumbai requests app.js
|
v
Nearest CDN Edge Server (Mumbai) <-- CACHE HIT: served in ~10ms, origin never contacted
| (only on a CACHE MISS)
v
Origin Server (e.g., in Virginia, US) <-- fetches once, then caches at the edge for ALL future
Mumbai requests
```

Internal Working

- A CDN is architecturally Chapter 7's cache-aside pattern applied at a **geographic** layer instead of an application layer — the "cache key" is the URL/asset path, and the "cache miss" fallback is a request to the origin server, exactly mirroring `ProductService.getProduct()`'s structure.
- CDNs are most effective for **static, infrequently-changing content** (Book 15A's compiled React bundle, images, videos) — content that changes per-request (a personalized API response) can't be meaningfully CDN-cached, which is why CDNs complement, not replace, Ch.7's application-level caching for dynamic data.
- Latency reduction is the primary benefit, not just load reduction: physics imposes a real speed-of-light floor on how fast a request can travel from Mumbai to Virginia and back — no amount of origin-server optimization can beat simply serving the content from a server physically closer to the user.

Real-World Example

Book 15A's React frontend, once built for production, is a textbook CDN use case — the compiled static JS/CSS/HTML bundle changes only on deployment, making it ideal for edge caching, while API calls (Book 15A, Ch.7's `apiRequest()`) still go to the origin/gateway (Book 16, Ch.6) for dynamic data.

Interview Answer

"A CDN caches static content on edge servers geographically distributed close to users, structurally the same cache-aside pattern as Chapter 7 but applied at a geographic layer — the URL is the cache key, and a miss falls back to the origin server. It's most effective for static, infrequently-changing assets like a compiled frontend bundle (Book 15A), not personalized dynamic API responses, which is why CDNs complement rather than replace application-level caching. The core benefit is latency reduction driven by physical distance, which no amount of origin-server optimization alone can overcome."

Cross Questions

- Q: How is a CDN structurally similar to Chapter 7's cache-aside pattern? → A: Both check a cache first (edge server vs application cache), and on a miss, fall back to a source of truth (origin server vs database) before caching the result for future requests.
- Q: Why can't CDNs effectively cache personalized API responses? → A: The content differs per user/request, so there's no shared cache key whose cached value would be valid for other users — CDN caching works best for content identical across many requesters.

Tricky Questions

- Q: Does using a CDN eliminate the need for application-level caching (Ch.7)? → A: No — CDNs handle static assets close to the user; dynamic, personalized, or frequently-changing data still needs application-level or database-level caching strategies, since it can't be meaningfully served from a geographic edge cache.

Coding Exercise

L1: Identify which parts of a typical web application (static assets vs API responses) are CDN-cacheable. **L2:** Design the CDN + origin architecture for Book 15A's React frontend paired with a Book 16 API Gateway. **L3:** Explain cache invalidation for a CDN when a new frontend version is deployed. **L4 (Interview):** Explain why a CDN's primary benefit is latency, not just load reduction. **L5 (Senior):** Design a caching strategy combining CDN (static), Redis (Ch.7, semi-dynamic), and no caching (fully personalized) across one application's endpoints. **L6 (Mastery):** Explain CDN cache invalidation strategies (versioned URLs/cache-busting vs explicit purge) and their trade-offs.

CHAPTER 9 — Asynchronous Communication & Messaging Patterns

Telugu Explanation

Book 17 లో Kafka ని deep గా నేర్చుకున్నాము. ఈ chapter HLD level కి zoom-out చేసి, ఎప్పుడు async messaging వాడాలో (decoupling, traffic spikes absorb చేయడం, long-running tasks) మరియు **queue vs pub-sub** architectural choice ని చూస్తుంది.

Professional English Explanation

Book 17 taught Kafka in deep detail. This chapter zooms out to the HLD level, examining **when** to use async messaging (decoupling, absorbing traffic spikes, long-running tasks) and the **queue vs pub-sub** architectural choice.

Diagram — Queue vs Pub-Sub at HLD Scale

```
QUEUE (point-to-point, ONE consumer processes each message):
  Producer -> [Queue] -> ONE of many Worker instances (competing consumers - for LOAD
DISTRIBUTION)
  Use case: image resizing jobs, email sending - work should happen ONCE, by ANY available
worker

PUB-SUB (Book 17's Kafka topics, MULTIPLE independent consumer groups each get every message):
  Producer -> [Topic] -> Consumer Group A (Notification)
                        -> Consumer Group B (Analytics)
                        -> Consumer Group C (Fraud Detection)
  Use case: "this happened" events that multiple independent systems each need to react to
```

Internal Working

- The architectural decision is: does exactly **one** consumer need to handle each message (queue — competing consumers, for distributing load across workers), or does **every** independent subscriber need its own copy of each message (pub-sub — Book 17's consumer groups)? Conflating these leads to real bugs — using a queue when pub-sub was needed means only one of several interested services ever sees each event.
- Async messaging at HLD scale primarily solves **traffic spike absorption**: a sudden burst of requests (e.g., a flash sale) can be accepted immediately by writing to a queue/topic, with workers processing the backlog at a sustainable rate afterward — this decouples "how fast requests arrive" from "how fast they must be processed," which a purely synchronous system (Book 16, Ch.3) cannot do.
- This entire chapter is the HLD-level justification for decisions Book 16 (Ch.1's pub-sub) and Book 17 made at the implementation level — recognizing "this needs decoupling/spike absorption/multiple reactors" during requirements clarification (Ch.1) is what should trigger reaching for this pattern in an HLD interview, before any Kafka-specific detail is discussed.

Real-World Example

A ticket-booking flash sale accepts all incoming booking *attempts* into a queue immediately (so the frontend never times out under load), while a fixed-size worker pool processes them at a sustainable rate — absorbing a traffic spike the synchronous booking logic alone (Book 19, Ch.3) couldn't handle at that instant.

Interview Answer

"Async messaging decouples request arrival from request processing, which is essential for absorbing traffic spikes a purely synchronous system can't handle — accepting work into a queue/topic immediately, then processing it at a sustainable rate. The key architectural choice is queue (competing consumers, exactly one worker processes each message, for load distribution) versus pub-sub (Book 17's Kafka model, every independent consumer group gets its own copy of every message, for multiple reactors to one event). Recognizing which shape a requirement needs — during initial clarification, not as an afterthought — is what should drive reaching for this pattern."

Cross Questions

- Q: What's the key difference between a queue and pub-sub in terms of message delivery? → A: A queue delivers each message to exactly one consumer (for distributing load); pub-sub delivers each message to every independent subscriber (for multiple systems reacting to the same event).
- Q: How does async messaging help absorb a traffic spike that synchronous processing can't? → A: It decouples the rate work arrives from the rate it's processed — a burst can be accepted into a queue/topic immediately, with workers draining the backlog at a sustainable, bounded rate afterward.

Tricky Questions

- Q: If a system needs both "distribute this work across many workers" AND "let three independent services react to it," can one messaging construct alone provide both? → A: Not directly with a plain queue — this typically needs pub-sub (a topic) with multiple consumer groups, one of which internally distributes its share of messages across a pool of worker instances within that single group (Book 17, Ch.4) — combining both shapes rather than choosing one.

Coding Exercise

L1: Classify 5 scenarios (email sending, order events, image resizing, chat notifications, payment processing) as needing a queue or pub-sub. **L2:** Design a traffic-spike-absorbing architecture for a flash-sale ticket booking system. **L3:** Design a system needing BOTH load distribution and multiple independent reactors for the same event. **L4 (Interview):** Explain the queue vs pub-sub distinction with a concrete example of each. **L5 (Senior):** Justify choosing async messaging over Book 16, Ch.3's synchronous REST for a specific operation, and vice versa. **L6 (Mastery):** Design the full async

architecture for a video-upload-and-transcode pipeline, addressing both load distribution and progress notification.

CHAPTER 10 — Reliability at Scale (Redundancy, Failover, Rate Limiting)

Telugu Explanation

Book 16, Ch.7 లో Circuit Breaker/Retry/Timeout ని ఒక్క service call level లో నేర్చుకున్నాము. HLD level లో అదే reliability thinking ని **whole-system** స్థాయికి extend చేస్తాము: **Redundancy** (single point of failure ఎక్కడా ఉండకూడదు), **Failover** (ఒక component fail అయితే automatic గా backup కి switch అవ్వాలి), **Rate Limiting** (system ని overload నుండి కాపాడాలి).

Professional English Explanation

Book 16, Ch.7 taught Circuit Breaker/Retry/Timeout at a single service-call level. At HLD level, we extend that same reliability thinking to the **whole system**: **Redundancy** (no single point of failure anywhere), **Failover** (automatic switch to a backup when a component fails), **Rate Limiting** (protecting the system from overload).

Diagram — Eliminating Single Points of Failure

```
BEFORE (single points of failure everywhere):
  1 Load Balancer -> 1 App Server -> 1 Database
      X               X               X      <- any ONE failure takes down the whole
system

AFTER (redundancy at every layer):
  2+ Load Balancers (active-passive or active-active)
    -> N App Server instances (Book 16's horizontally-scaled microservices)
    -> 1 Primary DB + N Replicas (Ch.6) with automatic FAILOVER on primary failure
```

Internal Working

- **Redundancy** means every layer of the architecture (load balancer, app servers, database) has more than one instance — a system is only as reliable as its least-redundant component; a beautifully redundant app-server tier behind a single, non-redundant load balancer still has one overall point of failure.
- **Failover** requires **automatic detection** of failure (health checks, Ch.2) plus an automatic promotion mechanism (e.g., a replica automatically promoted to primary on the original primary's failure) — a "failover plan" that requires a human to notice and manually intervene is not really failover at HLD-interview standards, since the goal is minimizing downtime, not just having a recovery procedure documented somewhere.
- **Rate limiting** (Book 16, Ch.6's `RequestRateLimiter`) at HLD scale protects the *system as a whole* from being overwhelmed — not just one client from abusing one endpoint, but preventing a traffic spike (organic or malicious) from cascading into the kind of overload failure Book 16, Ch.7's Circuit Breaker exists to contain *after* it's already happening; rate limiting is the preventive measure, Circuit Breaker is the reactive one.

Real-World Example

A production database cluster typically runs one primary with automatic failover to a synchronously-replicated standby — detected and promoted within seconds by a cluster manager, without requiring a human on-call engineer to notice and manually intervene during that critical window.

Interview Answer

"Reliability at HLD scale means eliminating single points of failure at every layer — load balancer, app tier, and database all need redundancy, since a system is only as reliable as its least-redundant component. Failover must be automatic, combining health checks (Book 12's Actuator concept, applied at infrastructure level) with automatic promotion of a backup, not a manual runbook. Rate limiting, already implemented concretely at the Gateway level in Book 16, is the preventive counterpart to Book 16 Ch.7's reactive Circuit Breaker — rate limiting stops overload before it starts, while circuit breaking contains the damage once a component is already struggling."

Cross Questions

- Q: Why is a system "only as reliable as its least-redundant component"? → A: Any single non-redundant layer (even if every other layer is fully redundant) remains a single point of failure that can take down the entire system.
- Q: What's the difference in role between rate limiting and Circuit Breaker (Book 16, Ch.7) at HLD scale? → A: Rate limiting is preventive — it stops overload before it happens; Circuit Breaker is reactive — it contains damage after a component is already failing/overloaded.

Tricky Questions

- Q: Does having a documented manual failover procedure count as "failover" at HLD-interview standards? → A: Generally not sufficiently — the expectation is automatic detection and promotion within seconds, since the whole point of failover is minimizing downtime, which a manual, human-in-the-loop process cannot reliably achieve during an active incident.

Coding Exercise

L1: Identify every single point of failure in a simple 3-tier architecture diagram. **L2:** Design a redundant version of that architecture, addressing each layer. **L3:** Design an automatic failover mechanism for a primary-replica database setup. **L4 (Interview):** Explain the difference between rate limiting and Circuit Breaker's roles in system reliability. **L5 (Senior):** Design a multi-region failover strategy for a globally-distributed system. **L6 (Mastery):** Design a full reliability strategy (redundancy + failover + rate limiting + Circuit Breaker) for a payment-processing system, explaining how each layer complements the others.

CHAPTER 11 — Microservices Patterns at HLD Scale

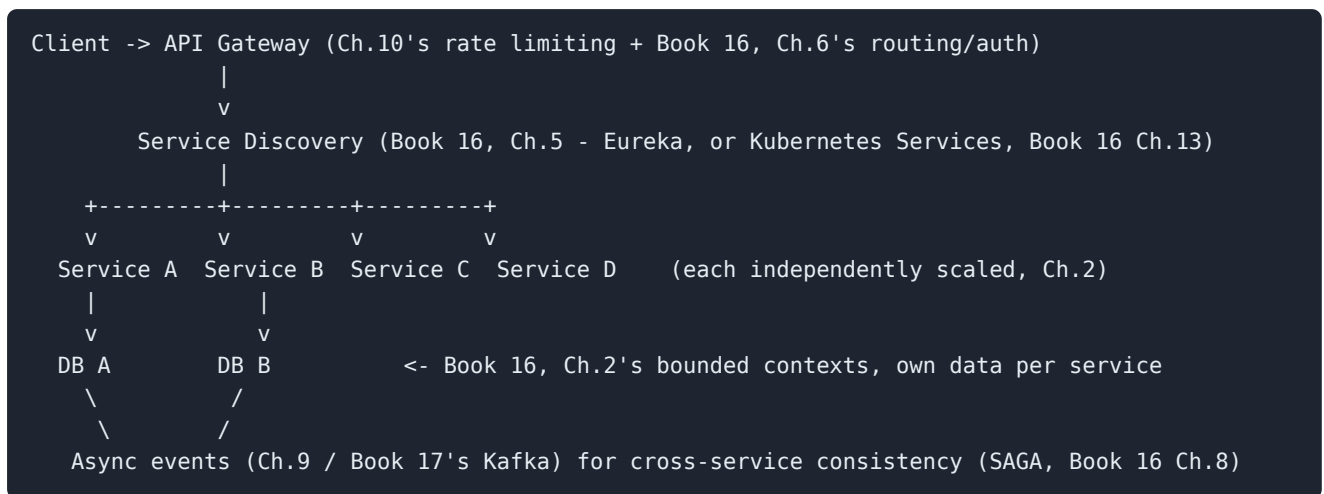
Telugu Explanation

Book 16 లో microservices patterns (API Gateway, Service Discovery, Config Server, SAGA, CQRS) ని Spring-specific code తో నేర్చుకున్నాము. ఈ chapter వాటిని **language-agnostic HLD vocabulary** గా recap చేస్తుంది — ఏ system design interview లోనైనా, ఏ tech stack తోనైనా, ఈ patterns apply చేయగలిగేలా.

Professional English Explanation

Book 16 taught microservices patterns (API Gateway, Service Discovery, Config Server, SAGA, CQRS) with Spring-specific code. This chapter recaps them as **language-agnostic HLD vocabulary** — applicable in any system design interview, with any tech stack.

Diagram — The Microservices HLD Reference Architecture



Internal Working

- At HLD-interview level, an interviewer doesn't need Spring-specific detail — they need the candidate to correctly identify **which architectural role** each component plays: a gateway (single entry point, cross-cutting concerns), a discovery mechanism (dynamic instance location), a data layer per service (bounded contexts, Ch.4-6 applied per-service), and an async layer for cross-service consistency (Ch.9) — the same shape regardless of whether it's implemented in Spring Cloud, Kubernetes-native constructs, or another stack entirely.
- **Monolith-first is often the right HLD answer**, not microservices-by-default — Book 16, Ch.1's monolith-vs-microservices trade-off applies directly here: an HLD interviewer proposing "design Instagram" from scratch is often better served by proposing a simpler initial architecture and explicitly discussing when/how it would evolve toward microservices as scale (Ch.3) demands it, rather than jumping straight to a fully decomposed microservices diagram.
- **CQRS and SAGA** (Book 16, Ch.8-9), recapped here as general HLD tools rather than Spring-specific code: SAGA answers "how do we maintain consistency across independently-owned data stores without distributed transactions," and CQRS answers "how do we serve a very different

read pattern than our write pattern" — both are specific, targeted answers to specific HLD symptoms, not default architecture choices.

Real-World Example

Most successful large-scale systems (Amazon, Netflix) didn't start as microservices — they started as monoliths and decomposed incrementally as team size and scale genuinely demanded it, exactly the trade-off-aware progression an HLD interview rewards over a default "everything is a microservice" answer.

Interview Answer

"At HLD scale, microservices patterns generalize beyond any specific framework: an API Gateway as the single entry point, a discovery mechanism for dynamic instance location, per-service data ownership (bounded contexts), and an async layer (queue or event stream) for maintaining consistency across independently-owned data without distributed transactions — which is exactly what SAGA (Book 16) provides. I don't default to microservices for every HLD problem — a monolith-first approach is often the right initial answer, with an explicit discussion of when and how the system would decompose as scale genuinely demands it, which is a more senior-level answer than jumping straight to a fully decomposed diagram."

Cross Questions

- Q: Why might proposing a monolith-first architecture be a stronger HLD interview answer than immediately proposing microservices? → A: It shows awareness of Book 16, Ch.1's actual trade-off — microservices add real distributed-system complexity that's only justified by genuine scale/team-size needs, not by default; explicitly discussing when decomposition would become necessary demonstrates deeper judgment.
- Q: What specific problem does SAGA solve at the HLD level, restated in framework-agnostic terms? → A: Maintaining consistency across multiple independently-owned data stores without a distributed ACID transaction, using local transactions plus compensating actions instead.

Tricky Questions

- Q: Does recognizing "this needs CQRS" mean every read operation in the system should go through a separate read model? → A: No — CQRS is justified specifically where read and write patterns diverge significantly enough to warrant the added complexity and eventual-consistency window; applying it universally across an entire system, including simple CRUD paths, is over-engineering, as Book 16, Ch.9 explicitly cautioned.

Coding Exercise

L1: Draw the microservices HLD reference architecture for a food delivery platform (reusing Book 19, Ch.9's domain). **L2:** Justify, for a small startup's MVP, why a monolith-first approach may be preferable to microservices. **L3:** Identify one part of a hypothetical system that genuinely needs SAGA, and one that would be over-engineered by adding CQRS. **L4 (Interview):** Explain the microservices HLD reference architecture without naming a single specific framework. **L5 (Senior):**

Design the migration path from a monolith to microservices for a growing e-commerce platform, in stages. **L6 (Mastery):** Explain how Book 17's Kafka fits into this HLD reference architecture as the async layer, in framework-agnostic terms.

CHAPTER 12 — Putting It Together: A Full System Design Walkthrough

Telugu Explanation

ఈ chapter ఇప్పటివరకు నేర్చుకున్న 11 chapters ని ఒక్క complete example తో — **URL Shortener** (బాగా known, కానీ ప్రతి concept ని touch చేసే) — synthesize చేస్తుంది, Chapter 1's 7-step framework ని పూర్తిగా apply చేస్తూ.

Professional English Explanation

This chapter synthesizes all 11 preceding chapters through one complete example — a **URL Shortener** (well-known, but touching every concept) — fully applying Chapter 1's 7-step framework.

The Full Walkthrough

STEP 1 - REQUIREMENTS:

Functional: shorten a long URL, redirect a short URL to the original, optional custom aliases, expiry

Non-functional: high read:write ratio (redirects >> creations), low redirect latency, high availability

STEP 2 - CAPACITY (Ch.3):

100M new URLs/month -> ~40 writes/sec. Assume 100:1 read:write -> ~4,000 reads/sec.

Storage: 100M URLs/month x 500 bytes x 12 months x 5 years = ~3TB over 5 years -> single DB feasible for a while,

but redirect READ latency at 4,000/sec strongly justifies caching (Ch.7) regardless of storage size.

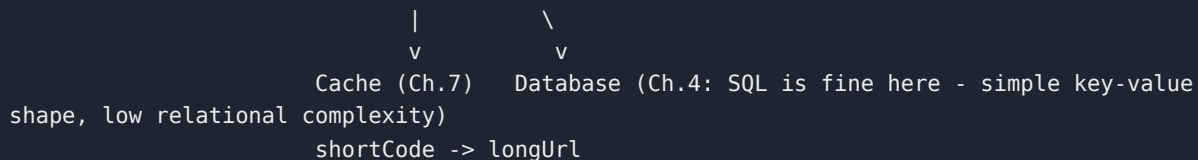
STEP 3 - API:

POST /api/urls {longUrl, customAlias?, expiryDate?} -> {shortUrl} (Book 12's REST design)

GET /{shortCode} -> HTTP 302 redirect to the original long URL

STEP 4 - HIGH-LEVEL COMPONENTS:

Client -> LB (Ch.2) -> App Servers (stateless, Book 14's JWT if auth needed)



STEP 5 - DEEP DIVE: Short Code Generation

Option A: Base62 encode an auto-incrementing DB ID -> simple, but reveals creation order/volume

Option B: Pre-generate a pool of random unique codes, hand them out from a dedicated service -> avoids the

auto-increment single point of contention at very high write volume (a real HLD trade-off to discuss)

STEP 6 - BOTTLENECKS:

The redirect path (GET /{shortCode}) is the hot path (4,000 reads/sec) -> cache-aside (Ch.7) is

the single highest-leverage optimization; a cache MISS should still be rare given popular URLs

follow a power-law distribution (a small fraction of URLs receive most of the traffic)

STEP 7 - TRADE-OFFS:

SQL (simple, ACID) vs a key-value store (Ch.4, better raw throughput for this exact access pattern) -

either is defensible; the interview should show you can articulate BOTH sides, not present one as "correct."

Internal Working

- This walkthrough deliberately demonstrates that **every earlier chapter's concept gets used to justify a specific decision**, not just mentioned — the capacity estimate (Ch.3) directly justifies caching (Ch.7) before caching is even proposed, which is the "numbers driving decisions" discipline emphasized since Chapter 1.
- The **short-code generation deep dive** (Step 5) is a deliberate example of Chapter 1's "deep-dive into 1-2 components" guidance — a URL shortener's interesting engineering problem isn't the

CRUD API, it's uniquely and efficiently generating short codes at scale without contention, which is exactly the kind of component an interviewer wants explored in depth rather than glossed over.

- The **power-law observation** in Step 6 (a small fraction of URLs receive most traffic) is what makes caching *especially* effective here — cache hit rates in real systems with this access pattern are often well above 90%, which is worth stating explicitly as a justification, not just asserting "we'll add a cache" without reasoning about why it will actually help this specific workload.

Interview Answer

"For a URL shortener, I'd walk through: functional and non-functional requirements first, revealing this is heavily read-dominated; capacity estimation showing redirect traffic at thousands of requests per second, which directly justifies a cache-aside layer; a simple REST API; a high-level architecture with a load balancer, stateless app servers, a cache, and a database; then a deep dive specifically on short-code generation, since that's the component with genuinely interesting engineering trade-offs, comparing auto-increment-based encoding against a pre-generated code pool. I'd close by naming the real trade-off between a simple relational store and a key-value store for this specific access pattern, without presenting either as objectively correct."

Coding Exercise

L1: Redo this full 7-step walkthrough for a different system: a pastebin/code-sharing service. **L2:** Identify which 1-2 components deserve the deep dive for a chat application, and justify your choice. **L3:** Estimate capacity for a URL shortener with 1 billion existing URLs and 10,000 redirects/sec, and state what that changes about the design. **L4 (Interview):** Deliver this entire URL shortener walkthrough, from memory, in under 15 minutes. **L5 (Senior):** Extend the URL shortener design to support click-analytics (tracking every redirect) without slowing down the hot redirect path. **L6 (Mastery):** Apply the full 7-step framework, live, to a system chosen at random by a peer, and defend every trade-off decision made along the way.

FINAL REVISION NOTES

- The HLD 7-step approach (Ch.1) is the backbone of this entire book — every chapter's concept exists to be *used* within that framework, not memorized in isolation.
 - Capacity estimation (Ch.3) is not a formality — its numbers (read:write ratio, storage growth) should visibly justify every major architecture decision that follows.
 - CAP theorem (Ch.5) reduces to a C-vs-A choice in practice, since Partition Tolerance is not optional in a real distributed system.
 - Caching (Ch.7) and CDN (Ch.8) are the same cache-aside pattern applied at the application layer and the geographic layer, respectively.
 - Reliability at HLD scale (Ch.10) generalizes Book 16, Ch.7's single-service resilience patterns to the whole system: redundancy, automatic failover, and preventive rate limiting alongside reactive circuit breaking.
 - Microservices patterns (Ch.11) are language-agnostic architectural roles, not Spring-specific mechanisms — the same shape applies regardless of tech stack.
 - Every HLD decision is a trade-off — a strong answer names the trade-off explicitly rather than presenting one option as objectively correct.
-



CHEAT SHEET

Concept	One-Line Summary
Vertical vs Horizontal Scaling	Bigger machine vs more machines; horizontal requires statelessness
Load Balancing	Round robin / least connections / consistent hashing; health-check aware
Capacity Estimation	Round numbers -> QPS, storage, read:write ratio drives decisions
SQL vs NoSQL	Relational integrity/ACID vs schema flexibility/horizontal scale
CAP Theorem	P is mandatory; the real choice is C vs A
Replication	Same data, multiple copies -> read scale + availability
Sharding	Different data, different servers -> write scale + storage limits
Cache-Aside	Check cache -> miss -> DB -> populate cache; invalidate (delete) on write
CDN	Cache-aside applied geographically, for static content
Queue vs Pub-Sub	One consumer per message vs every subscriber gets every message
Redundancy/Failover	No single point of failure; automatic detection + promotion
Rate Limiting	Preventive (before overload) vs Circuit Breaker's reactive (during overload)



INTERVIEW QUESTION BANK — System Design / HLD

Beginner

1. What's the difference between vertical and horizontal scaling?
2. Why does horizontal scaling require statelessness?
3. What's the purpose of capacity estimation in an HLD interview?

Intermediate 4. Explain CAP theorem and why Partition Tolerance isn't really optional. 5. Explain the difference between replication and sharding. 6. Explain cache-aside and why cache invalidation deletes rather than updates.

Advanced 7. Design a caching + CDN strategy for a read-heavy content platform. 8. Explain queue vs pub-sub and design a system needing both. 9. Design a redundant, automatically-failing-over 3-tier architecture.

Senior/Architect 10. Given a new, unfamiliar prompt, apply the full 7-step HLD framework live, including capacity estimation driving your architecture choices. 11. Justify a monolith-first approach for a given startup scenario, and describe the specific triggers that would justify migrating to microservices. 12. Design the complete data layer (SQL/NoSQL choice, replication, sharding, caching) for a system with a 100:1 read:write ratio and 5TB of yearly data growth.



CROSS-QUESTION ENGINE — Sample Chains

- Q: Why does Book 16's Eureka choose AP over CP? → A: An unavailable registry blocks all inter-service communication, which is worse than occasionally routing to a stale/dead instance. → Cross: What absorbs the consequence of that stale routing? → A: Circuit Breaker and Retry (Book 16, Ch.7) — the reactive reliability layer catching what the AP choice occasionally lets through.
 - Q: Why does a 25:1 (or 100:1) read:write ratio justify caching? → A: The overwhelming majority of load is reads, which a cache can absorb cheaply, leaving the database to handle only the much smaller write volume. → Cross: What happens if that ratio were reversed (write-heavy)? → A: Caching helps far less; sharding (Ch.6) and a write-optimized data store (Ch.4) become the higher-leverage investments instead.
-



CONSOLIDATED EXERCISES (All Levels)

- Apply the full 7-step HLD framework to 3 different systems (a chat app, a ride-sharing dispatch system, a news feed) from a blank page.
 - For each system, explicitly compute capacity estimates and use them to justify at least 3 architecture decisions.
 - Identify every single point of failure in a given architecture diagram and redesign it with full redundancy.
 - Mock-interview: have a peer pick a system at random and time a full HLD walkthrough in 30-40 minutes.
-



ONE-DAY REVISION PLAN (≈6 hours)

Time	Focus
0:00–0:45	Ch.1–2: HLD approach, scaling & load balancing
0:45–1:30	Ch.3: Capacity estimation drills
1:30–3:00	Ch.4–6: Data layer (SQL/NoSQL, CAP, replication/sharding)
3:00–4:00	Ch.7–8: Caching & CDN
4:00–5:00	Ch.9–10: Async messaging, reliability at scale
5:00–6:00	Ch.11–12: Microservices at HLD scale + full walkthrough



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1-2 — approach, scaling; solve one system's high-level diagram
2	Ch.3 — capacity estimation, deep practice with multiple systems
3	Ch.4-5 — SQL/NoSQL, CAP theorem
4	Ch.6 — replication & sharding, deep focus
5	Ch.7-8 — caching & CDN, deep focus
6	Ch.9-10 — async messaging, reliability at scale
7	Ch.11-12 — microservices recap + full mock HLD interview



FINAL MASTERY CHECKLIST

- ☐ I can apply the 7-step HLD approach to any new system, unprompted.
- ☐ I can perform capacity estimation and use its numbers to justify architecture decisions.
- ☐ I can correctly choose between SQL and NoSQL for a given access pattern.
- ☐ I can explain CAP theorem and correctly classify real systems as CP or AP.
- ☐ I can design replication and sharding strategies and choose an appropriate shard key.
- ☐ I can design a cache-aside strategy and explain invalidation correctly.
- ☐ I can distinguish queue from pub-sub and choose correctly for a given requirement.
- ☐ I can design a fully redundant architecture with automatic failover.
- ☐ I completed a full 30-40 minute mock HLD interview under time pressure.

Next: `22_System_Design_Case_Studies.md` — Book 22, applying every concept from this book to 5 complete, real-world system designs (WhatsApp, Netflix, Hotstar, Instagram, Rate Limiter).